The Distributed Run Manager
A Parallel Processing C++ Library

Brian P. Harrawood*

Abstract

*The **DistributedRunManager** (DRM) is an application level tool for distributing a computing task among a number of computer systems for parallel processing. It is a small collection of C++ Classes which can be incorporated into a C++ project, or used to create a wrapper/driver for any program launched from a* shell *program.*
The model for distribution is an unbounded client-server in which the clients perform the computing tasks and the server maintains the state of the overall progress. The model is tolerant of individual processes (clients) starting and stopping before the entire run is complete.
*The DRM self-configures at runtime using a novel C++ class, the **ElectionManager**. This class cooperatively selects a server from among the processes which were started and ensures agreement going forward on which process in the flight acts as the server.*

Introduction

The `g4DistributedRunManager` described in [1] is an early precursor to the DRM. The current DRM is a nearly complete rewrite which decouples the code from the **GEANT4 Toolkit** [2]. It also adds a new class, the `ElectionManager` which permits DRM enabled applications to self-organize at runtime and select a process from among themselves to manage the state of the overall task.

The DRM is open source released under GPLv2 and is available on GitHub. User documentation and sample code are provided there. This document is not intended as a user guide, but rather an in-depth examination of the design of the library.

A quick note on the nomenclature used. The word *run* is used in the sense of a "production run" to indicate an entire task. For instance, it might refer to performing a simulation or an estimation. The DRM splits a run into multiple *processes* (i.e., Linux processes). The DRM harnesses multiple processes and uses them to perform a run.

Design Overview

A distinction is often made between *parallel processing* and *distributed processing*[3]. This difference regularly hinges on whether the run is accomplished using a single node (i.e., parallel) or multiple nodes connected via a network (i.e., distributed). The DRM is designed to use a blend of both approaches, employing multiple processes on a single node and/or multiple nodes. It is not designed for networks which are widely (globally) distributed.

The DRM model for maintaining the state of the run is the client-server model. In particular, it is a Single Server Multiple Client System (SSMCS) [4]. In this system, the server provides *work-tickets* to the clients for completion. The system is unbounded in the number of clients it can support, and clients are free to come and go as the run evolves.

The server process keeps track of which tickets are completed, which are running, and which are ready-to-run. Using message passing, a client process requests a piece of the run (i.e., a ticket) from the server. It completes that portion of the run and then reports it back to the server.

The DRM server should not be confused with the term server used for a persistent process such as the `httpd` server. The DRM server is created at the beginning of a run and exits when the run is complete. There are no `daemon` processes used.

The DRM server is not just the keeper of the state. It also has all the code that a client process has after all, and so it can also process tickets. A DRM application will run as a stand-alone process without the need of running additional clients. In stand-alone mode, the server asks itself for tickets and processes them.

The network bandwidth requirements on communications between clients and server are modest. Each client starts with all the data needed to do any piece of the task; the server only needs to communicate to the client *which* task to do next. The client handles the output on completion of the task. The network protocol rarely requires a network message of more than 512 bytes.

*Corresponding author: brian.p.harrawood@gmail.com

Designating the server - Election Management

The client-server algorithm requires that there is one and only one server. To automate this, the DRM creates an `ElectionManager` object which chooses a server from the running processes. This runtime election removes the need for a mechanism to do *a priori* designation of clients and server.

The distinction in the DRM between client and server processes is handled at runtime with logical operators. The single code base can execute in either mode. The DRM process starts out as neither client nor server. The `ElectionManager` assigns one role or the other to the process and that role is then immutable.

When a process starts, it broadcasts to the network, seeking a server process. If a server answers, the process connects as a client. If no server answers and no other broadcasts are detected, the process becomes the server.

But, if there are other processes searching for a server and there is no server, an election is held among the candidates to select one. (All processes in the *flight* are eligible candidates.) The cooperatively selected process is "elevated" to server and the remaining processes attach as clients.

Task Granularity - Tickets

The pieces of the run distributed to the clients are described by *tickets*. A run is divided into pieces which can be uniquely mapped to tickets. This is sometimes referred to as the granularity of the distribution. Each ticket completely describes a piece of the run to be executed within a single process. If the overall run is conceived of as a logical loop, then each ticket is an iteration of that loop.

Generally, each ticket uses different input data to generate its own output data: loosely, a variation of the Single Instruction Multiple Data (SIMD) processing algorithm. The output data are written to file by the client process. At the end of the run the DRM relies on a shared file system (e.g., NFS) to collate these files.

The tickets are dynamically distributed to clients. As a client comes free, it will be assigned the next ticket by the server. This allows DRM-enabled applications the flexibility to run with any number of clients. Additionally, the number of clients may change *during* the run and the DRM will gracefully accommodate.

Extensibility Considerations

An effort is made to split the DRM into classes which are opaque and classes which "hook" into the application. The opaque classes (`DistributedRunManager` and `ElectionManager`) offer little configuration while the other classes (`TicketManager` and `Ticket`) are abstract base classes which require application level coding.

Both the `DistributedRunManager` and the `ElectionManager` contain messaging details (e.g.; protocols, socket handling) and run logic (e.g.; client and server mode selection) which have been abstracted from the application programmer. It is anticipated that these details can be safely forgotten.

However, the `TicketManager` and `Ticket` need flexibility to accommodate the specifics of a distribution. The details of dividing a task into pieces are handled in these classes. Making them base classes provides the application programmer the flexibility to create their own new, derived classes.

Design Constraints

The DRM is not designed as a general purpose problem solving tool. It was designed with particular types of problems, software environments, and hardware in mind. It is important to understand these constraints before applying it to your problem.

Problem Characteristics The DRM is only useful for solving a particular class of problems. The problems must be "parallel-izeable" in nature. In other words, they must require many repetitive iterations, each of which is independent of the any other, such that iteration $i + n$ does not rely on output from iteration i. Each of these iterations will be a DRM ticket.

The number of tickets must be calculable (there is no provision for iterating to a limit) and must be known at the start of the run. The number of tickets is also a factor in the overall throughput. This has to do with the granularity of the distribution.

A granularity which is too fine (too many pieces) may lead to ticket execution times comparable to the transmission time of a ticket. A granularity which is too coarse (too few pieces) limits the expanse of the distribution and negates the performance.

In practice, a DRM ticket is expected to be very small, e.g., just an integer or short string. The time to pack/unpack and transmit a ticket needs to be much, much less than the time to execute the ticket or the overall time spent on a run is spent mostly passing messages.

Software Environment The DRM is an application level tool. It has no facility for starting, stopping, or reporting on system processes. To effectively

use a DRM-enabled application, these "meta" activities must be managed by another agent: either manually at the command-line, or scripting methods or an executive manager such as slurm (see `https://slurm.schedmd.com`).

Although DRM applications run optimally on a managed Linux cluster, they are still useful without one. On a single node, multiple instances of the application can be run one-by-one from a command line to improve throughput. Multiple nodes on a local network can be used in concert as a Beowulf cluster [5].

The Beowulf cluster technique requires network communication between nodes, shared user permissions, and a shared file system. These are all standard on a typical Linux network which has home or data directories shared. In this environment, a user starts the application a number of times on one node, then logs onto another node and starts more there. The applications communicate between nodes with no further intervention. In this situation, the management of resources (CPU cores or GPU's) is left to the user.

A managed cluster has the user permissions, local network and networked storage systems already in place. A single DRM-enabled application can be submitted as an *array-job* with a cluster job scheduler. The cluster management software handles all resource allocation and the DRM handles the client-server designation among the processes.

Hardware There is no need for cooperating DRM nodes to be homogeneous. As long as the application can be built and run on a platform, it is eligible for inclusion. There is nothing about the design which requires a particular platform: e.g., big or little endianess, operating system dependency, etc. However, it has to-date only been build and tested on Linux systems. No barriers are foreseen in porting the code to other platforms.

The network offers more subtle constraints. Nodes must be reachable "locally"; i.e., on the same subnet. Firewall security may need to be relaxed. The `ElectionManager` does a limited broadcast to only the local network. Nodes on the other side of a router are not within reach of this broadcast.

The network must also be low latency. The `ElectionManager` relies on fixed timing assumptions to determine if other nodes are participating in the run. Very high latency may cause problems with this feature. This will not be an issue in a well functioning ethernet network.

Implementation

The DRM is written in C++ using the C++ Standard Library and makes extensive use of Inter-Process Communication (IPC) mechanisms such as sockets, signals and pthreads. These features are part of the Posix Standard (`https://posix.opengroup.org`) and are available in all conforming operating systems. Library dependencies include `libz`, `libpthread`, and `libcrypto`, a part of the OpenSSL Project (`https://www.openssl.org/`).

The DRM is implemented with a small number of C++ classes: `DistributedRunManager`, `TicketManager`, `Ticket`, and `ElectionManager`. The implementation discussion is organized using these classes.

DistributedRunManager

The programming interface class for the DRM is the `DistributedRunManager`. This class is only created once per application. It accepts input from the application which includes a network port number and a network key. These distinguish a "run" and all processes in the run must agree on their values.

The network port number is the port which the DRM will use for network communications. On Linux systems it is a number between 0 and 65535. Ports 49152-65535 are available for private use and the DRM should use a port number in this range.

The network key is simply a string passed between processes as a handshake. If the handshake doesn't agree, communications are ended.

When the `DistributedRunManager` class is first instantiated it is neither a client nor a server. It creates an `ElectionManager` to determine which mode it will use going forward. The `ElectionManager` returns either the address of the server or informs the process that it is the server.

Server initialization To communicate with clients, the server starts a dispatcher. This is a collection of threads which communicate with clients. It starts with the server creating a thread which listens on the network port for connections. This thread *blocks* while waiting for connections and persists for the life of the `DistributedRunManager`.

When a client connects, the listen thread creates a new thread specifically for that client's communications. A client communication thread is created for every connected client. These threads *block* while waiting for client messages and persist only as long as the client is connected.

Meanwhile, the main thread returns to the application.

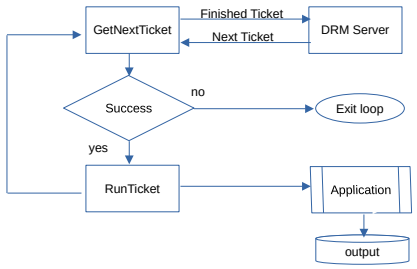


Figure 1: *The RunTickets loop. The next ticket is determined by the server and is communicated to clients. Clients run the ticket with the application and return the completed ticket to the server. The application handles the output.*

Client initialization The client mode of the DRM is not as complex. The client connects to the server, it exchanges the handshake, loads the parameters for the run, and then requests a ticket and blocks (waits) for the server to send some work.

Network Protocol The entire network protocol is comprised of three commands, each initiated by the client:

- Handshake
- Parameters
- Next Ticket

The client's first message to the server is it's handshake string. Part of this string is the *network key*. This is a string determined by the application developer. Each application needs a unique string to prevent DRM-applications from colliding. If the server and client handshakes disagree, communications are ended.

Next the clients request the input parameters for the run. Parameters are input to all processes from the command line when they start and it is a temptation to allow each to run with its own input. However, if parameters are not perfectly uniform, the clients' outputs are inconsistent. So the server process provides the input to the clients guarantying consistency across all clients.

Finally, clients request work tickets. This request also provides the server with the client's last completed ticket. When the server is finished with the tickets, it sends an **EndOfTickets** message and the client exits.

Running Tickets The DRM takes control of the application when a call is made to the `DistriutedRunManager.RunTickets()` method. It is called once and it retains control of the application until all tickets have been processed.

The `RunTickets()` method works the same in both server and client modes of the DRM. It simply requests the next ticket and runs it, until all the tickets have been handled. The request for the next ticket employs different code in the server than it does in the client, but the end-results are the same.

The `RunTickets()` method returns control to the application after all tickets have been completed.

TicketManager

The `TicketManager` is an abstract base class designed as a helper to the `DistributedRunManager`. This class is only created once per application. Each DRM must have a registered `TicketManager` in order to call `RunTickets()`.

The `TicketManager` is the base class from which the application developer must derive the specific class used in their application. This class is primarily responsible for incrementing and executing tickets. Since the `Ticket` class is unique to the application, so must be its manager.

The `RunTickets()` method described above, runs a loop which operates on a `Ticket`. Schematically, it gets the next ticket and then executes the ticket until all the tickets are completed.

The retrieval of the next ticket is *always* done in the server. In the server, the `TicketManager` `IncrementTicket()` gets the next ticket which is returned to the client.

Then the `HandleTicket()` method passes execution of the ticket to an application function using a callback. This works identically for both server and client.

When creating the `TicketManager` the constructor requires a callback function and callback argument, e.g:

```
catlib::DRMCallbackFunc_t callback_func =
    run_the_ticket;
MyTicketManager ticketManager(callback_func,
    &myApp);
```

The callback function with its argument is called when the `HandleTicket()` method is invoked, e.g.:

```
int MyTicketManager::HandleTicket(catlib::Ticket
    * ticket)
{
  MyTicket * my_ticket = (MyTicket *)ticket;

  int tckNumber = my_ticket->GetTicketNumber();

  CallbackFunc_(tckNumber, CallbackArg_);

  return(0);
}
```

The callback function in turn passes execution to the application. The application runs the iteration

described by the ticket and generates output. When the callback returns, execution is passed back to the DRM.

```
void run_the_ticket(int ticket, void * arg)
{
  MyApp * my_app = (MyApp *) arg;

  my_app->doCalculation(ticket);

  return;
}
```

This repeats until the tickets are exhausted.

Ticket

The `Ticket` is another abstract base class designed as a helper, but for the `TicketManager`. This class is not a singleton. The application must provide a *derived* class with the appropriate data structures to define a ticket.

A `Ticket` is primarily a container for the information the application needs to run an iteration. Often this is simply an integer.

It is also responsible for packing and unpacking that information for network transmission - as a string. Obviously, this is a *shallow* transfer of the ticket since true marshaling is not supported.

ElectionManager

The `ElectionManager` is not called directly from the application. A single instance is created by the `DistributedRunManager` when it starts.

The election process requires its own unique network port. If multiple `ElectionManager`'s are started on the same host only one of them will be able to open the network port. That one is the *Network* instance. All subsequent instances *on that host* for this run are *Local* instances.

Network ElectionManager If opening the network listening socket is successful (i.e., the *Network* instance), the `ElectionManager` broadcasts inquiry messages to the network repeatedly for `TMOUT_INQUIRY` seconds. If a server is available, it responds with its IP address. The listening thread passes the information back to the `ElectionManager` main thread where it is available to the DRM.

If no servers answer during `TMOUT_INQUIRY` seconds **and** no inquiries arrive from other processes, then this process becomes the server.

However, if no server answers after `TMOUT_INQUIRY` seconds, and there are other processes seeking the server, then an election is started. The listening thread has been collecting information from other

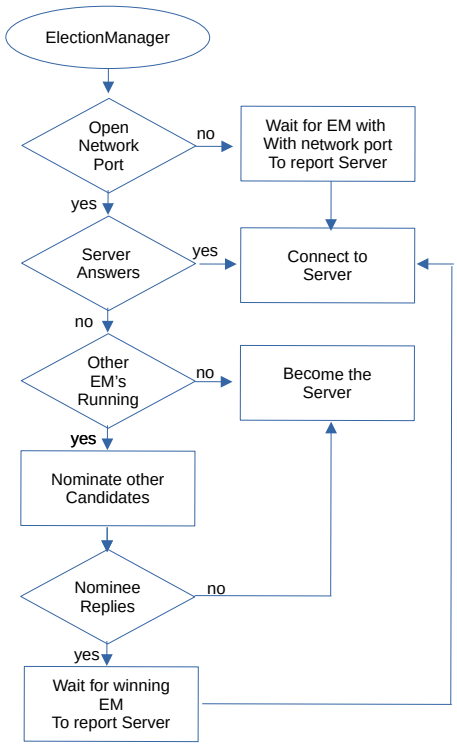


Figure 2: *The ElectionManager negotiates asynchronously with other Election Managers running on the host and/or network to determine which will become the server.*

`ElectionManager` hosts broadcasting for a server and a candidate list is made of these hosts.

The candidate list is sorted by IP address. All IP addresses greater than "mine" are put into a nominee list and a nomination message is sent to each nominee. Meanwhile, every nomination message received is answered with a nomination accepted message.

After `TMOUT_ELECTION` seconds the list of hosts accepting their nomination is examined. If the list is empty, the process sets itself as server and answers all pending and new broadcast messages with its IP address. If the list is not empty, an inquiry broadcast is repeated until the elected server reports back.

Local ElectionManager Processes without access to the network port (i.e., the *Local* instance) open a *unix* socket with which they can communicate with the *network* process. When the *network* process has determined the server's IP address, it informs all *local* processes.

Post Election After a server is selected, it's `ElectionManager` responds to new broadcasts on the election port with it's IP address and all processes starting after the election will be directed to the same server.

Caveats The `ElectionManager` uses asynchronous communication on the network. It is impossible to plan for for every situation which may arise. Network or host configuration issues or extreme load conditions may cause the election to fail.

If problems persist, the `DisributedRunManager` provides constructors for specifically starting server or client objects and then the `ElectionManager` is removed from the process.

The election also adds latency to each process. Approximately 4 seconds is required for the selection to complete. This is negligible for a typical DRM-application which may be running for hours or 10's of hours.

Conclusion

Much effort has been devoted to distributed computing [3] with the advent of less expensive computers and more reliable network connections. However, applications must be modified to leverage the efficiencies of these distribution systems. The DRM addresses this problem.

Not all applications are candidates for distributed computing. However, for those problems which can benefit from distributed computation, the DRM offers a reusable tool which provides a single interface to the distribution problem.

The DRM interface hides the message passing, client-server selection process, client enrollment, and error handling of the distribution. It scales without modification from a single process to a High Performance Compute Cluster.

The DRM has been used for many years, for varied application types, and in many different distribution environments. It is a robust and flexible solution for transforming a serial application into a distributed application.

Acknowledgments

The DRM was first begun in the 2003-2004 time frame as a solution to increase throughput of Monte Carlo analyses, specifically for the NSECT program [6]. Its slow development over the years since then was made possible by Duke University's Radiology Department.

Originally it ran on single hosts, was migrated to a Beowulf cluster, then to the Duke University's compute cluster and finally to the Duke Radiology Compute Cluster, a High Performance GPU based cluster. As the years passed, the research projects supported by the DRM were numerous and varied, branching into image processing, synthetic image generation, and general purpose distributed calculations.

The author is grateful for the support and patience of the many researchers he worked with during these decades. Without the support of Rai Labs Advanced Imaging (`https://rai.labs.duke.edu`) and the Center for Virtual Imaging Trials (`https://cvit.duke.edu`) this work would never have been completed.

References

[1] Brian P Harrawood et al. "Geant4 distributed computing for compact clusters". In: *Nuclear Instruments and Methods in Physics Research Section A: Accelerators, Spectrometers, Detectors and Associated Equipment* 764 (2014), pp. 11–17.

[2] Sea Agostinelli et al. "Geant4—a simulation toolkit". In: *Nuclear instruments and methods in physics research section A: Accelerators, Spectrometers, Detectors and Associated Equipment* 506.3 (2003), pp. 250–303.

[3] Md Firoj Ali and Rafiqul Zaman Khan. "Distributed computing: An overview". In: *International journal of advanced networking and applications* 7.1 (2015), p. 2630.

[4] Ramchandra Phawade, Tephilla Prince, and S Sheerazuddin. "Verification of Unbounded Client-Server Systems with Distinguishable Clients". In: *arXiv preprint arXiv:2602.23054* (2026).

[5] Donald J Becker et al. "BEOWULF: A parallel workstation for scientific computation". In: *Proceedings, international conference on parallel processing*. Vol. 95. 1995, pp. 11–14.

[6] Carey E Floyd Jr et al. "Neutron stimulated emission computed tomography of stable isotopes". In: *Medical Imaging 2004: Physics of Medical Imaging*. Vol. 5368. SPIE. 2004, pp. 248–254.